NEXUSQ GLOBAL · ENGINEERING HANDOFF

Video box and site help chat

Video box and AI chatbox: specifications, limitations, future plan, porting guide, and full source.

Date: 19 August 2026 (chat composer update)
Source repo: github.com/aryanvbabu/NexusQ
Live parent site: NexusQ Global (Next.js 16, App Router)
Purpose: reuse the same widgets on another live website after approval (target: AuditionQ or any NexusQ product site)

1. What we built on NexusQ Global

Two visitor-facing widgets were added to the NexusQ Global marketing site. They are independent: the video box can ship without the chat, and the chat can ship without the video box.

Widget	What the visitor sees	Shipped behaviour
Casting video box	Hero "Live audition / AuditionQ" monitor with looping BTS video	Click opens https://www.auditionq.com/ in a new tab
Site help chat	Fixed bottom-right **Help** button → assistant panel	Answers only NexusQ website questions; refuses general knowledge

The old floating **Website Tour** button (`EcosystemGuide`) was removed. Navbar **Guide me** (spotlight tour) is separate and was left in place.

Commit on main that introduced the chat: dc05e64. Video-box click-through is a later local change described in this document.

2. Table of contents

3. Video box — specification

3.1 Purpose

A compact “casting monitor” that proves the live product with motion, then sends the visitor to that product in one click. On NexusQ it advertises AuditionQ.

3.2 Functional spec

Item	Spec
Component	`app/components/CastingMonitor.tsx`
Styles	`app/globals.css` classes `.nq-casting-monitor*`
Placement	Hero: desktop right of copy; mobile below copy
Primary source	`public/videos/casting-bts.mp4`
Fallbacks	Three Mixkit/Pexels CDN MP4 URLs if local file fails
Poster	`public/scenes/casting-audition.webp`
Playback	autoPlay, muted, loop, playsInline; preload metadata
Click	Whole bezel is an `<a>` to https://www.auditionq.com/ (`target="_blank"`, `rel="noopener noreferrer"`)
Reduced motion	If `prefers-reduced-motion`, show poster image only (no autoplay)
Error handling	On video error, try next source; if all fail, poster only
Chrome sizes	Bezel width 340 / 360 / 400px; screen height 190 / 210 / 228px
Chrome chrome	REC dot, “Live audition”, AuditionQ badge, scanline overlay, caption “Visit AuditionQ”
Hover / focus	Cyan glow and 2px lift; video/img `pointer-events: none` so clicks hit the link
A11y	`aria-label="Visit AuditionQ"`; keyboard-focusable as a link
Dependencies	React; `framer-motion` only for `useReducedMotion()`

3.3 Non-goals (current)

- No sound, controls, fullscreen, or captions.
- No in-page product iframe — destination is the live product URL.
- Not a general video player; it is a branded CTA with motion.

3.4 Video box limitations

Limitation	Impact

Muted autoplay only	Browsers block unmuted autoplay; there is no volume control
CDN fallbacks	Third-party URLs can disappear or be the wrong brand on a second site
No captions / transcript	Not a complete media alternative for accessibility
No pause-when-offscreen	Video may still decode after the user scrolls away
Opens a new tab	Some products may want same-tab navigation
Single destination	Cannot deep-link to a logged-in dashboard without changing the URL
Local MP4 size	A large file hurts first load on mobile
Reduced-motion users	See a still only; the click still works

4. AI chatbox — specification

> **Product name on site:** NexusQ Assistant (floating **Help** button).
> **What it is:** a website-only guide that retrieves NexusQ facts and composes a reply shaped to the question (not one fixed paragraph per topic).
> **What it is not:** not ChatGPT, not a general AI, not live human support, not an AuditionQ account helper.

4.1 Purpose and scope

The assistant exists so a visitor can ask "what is this site?", "is AuditionQ live?", "how do I partner?", or "how do I sign in?" and get a short, honest answer plus a link. It must stay inside published NexusQ website facts. If the question is about weather, homework, coding, news, or any other general topic, it must refuse.

In scope	Out of scope
NexusQ Global identity; Live vs Vision vs Exploration; named products; how to use this website; partner form; email; sign-in/sign-up on this site; privacy/terms placeholders; theme; the clickable hero video box	General knowledge; writing code; AuditionQ in-app support (casting workflows, billing, password reset on auditionq.com); legal advice; invented metrics, launch dates, or office address

4.2 Architecture spec

Layer	File	Spec
UI	`app/components/SiteHelpChat.tsx`	Client widget: Help FAB, panel, bubbles, suggestion chips, input, POST to API
HTTP API	`app/api/help-chat/route.ts`	`POST /api/help-chat` only. No GET. No streaming.
Engine	`lib/site-help.ts`	Fact retrieval + question-shaped composer (yes/no, how, where, list). Skips facts already said.
Optional paraphrase	`lib/site-help-paraphrase.ts`	If `GROQ_API_KEY` or `OPENAI_API_KEY` is set, rewrite from those facts only. Otherwise unused.
Mount	`app/layout.tsx`	One instance on every route (home, login, partner, privacy, terms)

Runtime: Next.js App Router on the same origin. **No database.** Default path needs no API key (CPU compose only). Optional Groq/OpenAI paraphrase still may not invent facts. Cost per message is CPU, plus provider cost only if a key is set.

4.3 UI / UX spec

Element	Specification
Launcher	Fixed bottom-right (16–24px inset), z-index 80, cyan pill, pulse dot + "Help"

Open panel	Width min(22.5rem, viewport − 2rem); height min(32rem, 100dvh − 5.5rem). Launcher hides while open.
Header	Title “NexusQ Assistant”, subtitle “Website help only”, close (X)
Welcome	One assistant message explaining website-only scope + 4 suggestion chips
Default chips	What is NexusQ Global? · Is AuditionQ live? · How do I partner with you? · How do I sign in?
User bubble	Right-aligned, cyan
Assistant bubble	Left-aligned; optional link under the text; new suggestion chips after each reply
Loading	Shows “Thinking…” until the API returns
Input	Placeholder “Ask about this website…”; maxlength 500; Send disabled when empty or loading
Keyboard	Enter sends; Escape closes; focus moves to the input on open
Theme	Uses `--nq-*` tokens so light and dark both stay readable
Motion	Framer Motion panel enter/exit (~180ms)

4.4 API spec

Request — `POST /api/help-chat`, `Content-Type: application/json`

```
{
  "message": "Is AuditionQ live?",
  "history": [
    { "role": "user", "content": "What is NexusQ?" },
    { "role": "assistant", "content": "NexusQ Global is a parent company..." }
  ]
}
```

Field	Rule
message	Required string, trimmed, 1–500 characters
history	Optional array; last 8 items kept; each content clipped to 500 chars; role must be user or assistant

Success (200)

```
{
  "answer": "AuditionQ is NexusQ Global's live flagship product...",
  "link": { "label": "Visit AuditionQ", "href": "https://www.auditionq.com/", "external": true },
  "suggestions": ["Are any other products live?", "How do I partner with NexusQ?"]
}
```

Errors

Status	When
400	Empty message, or message longer than 500 characters
500	JSON parse / unexpected server error

4.5 Fact composer spec (current — 19 August 2026)

Replies are **not** a single canned paragraph per topic. Knowledge is stored as short facts. The engine picks facts that match the question type, then stitches a short answer.

1. Trim the message. Empty → prompt the visitor to ask about the website (varied wording).
2. Short greeting / thanks → varied welcome or acknowledgement, no retrieval.
3. Follow-up lines ("what about…", "tell me more…", ≤10 words) are expanded with the previous turn.
4. Detect intent: `yesno`, `how`, `where`, `why`, `list`, `what`, or `general`.
5. If off-topic and no site term → refuse (varied wording).
6. Score topics by keyword/title. Prefer intent-tagged facts (how-questions get how/where facts first).
7. Yes/no about AuditionQ live → a short "yes" lead + 1 supporting fact. Vision products → a short "no" lead.
8. How/where questions use at most 2 facts; broader what/list questions up to 3.
9. Facts already present in earlier assistant messages are skipped. If none remain: say that topic is covered and offer a follow-up.
10. Optional: `paraphraseSiteHelp` may rewrite the composed reply from those facts if a Groq/OpenAI key is set (4s timeout, then fall back to compose).

Site terms include product names (NexusQ, AuditionQ, FurSure, RideQ, CaringMinds, Onakkodi), partner/login/privacy/terms, and phrases such as "this website". Off-topic examples: weather, recipes, jokes, homework, sports, crypto, "write python/javascript/code".

4.6 Knowledge coverage spec (current)

Topic id	Visitor can ask about
what-is-nexusq	Who NexusQ is / what this website is
how-to-use-site	Nav, sections, Guide me, clicking the video box
auditionq	Live flagship, auditionq.com, monitor click
ecosystem	Named platforms and their status
live-vs-vision	Meaning of Live / Vision / Exploration
fursure, rideq, caringminds, onakkodi, future-ai	Each vision/exploration product — not launched
partner / email	Partner form and admin@auditionq.com
login	This site's sign-in/up (not AuditionQ product login); no Settings page
privacy / terms	That those pages are placeholders
trust / theme / vision-company	Honesty rules, light/dark, why the company exists

4.7 Honesty / safety spec

- Only AuditionQ may be described as Live.

- Vision and exploration products must not get store, download, or launch actions.
- No fabricated metrics, testimonials, customer logos, phone number, or street address.
- Legal pages must be described as placeholders until lawyer-reviewed copy exists.
- When public site copy changes, `lib/site-help.ts` must be updated in the same change.
- The assistant must not claim to be a large language model or to browse the web.

4.8 Privacy, security, performance spec

Item	Specification
PII	Chat does not ask for passwords. Messages are not written to the database. They exist in browser state and in the single POST body.
Auth	Endpoint is public (same as the marketing site). No login required to ask.
Rate limit	None in v1.
Secrets	None required. Optional `GROQ_API_KEY` or `OPENAI_API_KEY` only for paraphrase; still facts-only.
Latency	Local compose is typically tens of milliseconds. Optional LLM adds up to ~4s then falls back.
Availability	If `/api/help-chat` fails, the UI shows an error bubble and keeps the panel open.
Languages	English keywords and answers only.

4.9 Behaviour matrix (expected replies)

Visitor says	Expected behaviour
Hi / hello	Welcome + default chips
What is NexusQ?	Parent-company answer + Home link
Is AuditionQ live?	Short yes + live/flagship fact + link to auditionq.com (not the full AuditionQ essay)
How do I open AuditionQ?	URL and/or “click the homepage video box”
What products do you have?	Ecosystem list with Live/Vision/Exploration — not the company-overview paragraph
Is FurSure / RideQ live?	No — vision, not launched
How do I partner?	Partner form path + admin@auditionq.com
How do I sign in?	/login on this site; not AuditionQ product login
What’s the weather? / write my homework	Refuse; stay website-only
Thanks	Short acknowledgement
Unrecognised site-ish question	Fallback: try NexusQ / AuditionQ / partner / sign-in

5. AI chatbox — limitations

These limits are part of the product spec. Do not promise ChatGPT-quality answers when porting this widget.

5.1 Intelligence and accuracy

Limitation	What that means
Not a general LLM by default	Without a provider key it scores keywords and composes from facts. Unusual wording or typos can still miss.
No deep reasoning	It cannot debug an account or plan multi-step work. It only rearranges published site facts.
No live browsing	It cannot read auditionq.com, email, or a CMS at request time.
Knowledge can go stale	If marketing copy changes and `site-help.ts` is not updated, answers are wrong.
Two-topic merge	Close-scoring topics may both contribute facts; the reply can mix two subjects.
Follow-up quality is basic	Short “what about / tell me more” lines use prior context; long mixed questions may still miss.
Optional LLM can still fail closed	If Groq/OpenAI errors or times out, the composed fact answer is returned instead.

5.2 Scope and product honesty

Limitation	What that means
Website facts only	Cannot help with AuditionQ casting tools, billing, or password reset inside the live product.
No human agent	No live chat queue. The panel can only point at /partner or email.
Off-topic list is finite	Some general questions may still get a weak site match instead of a clean refuse.
Must not invent	Will not (and must not) create metrics, launch dates, prices, or an office address.
Legal copy is placeholder	Privacy/terms answers say the pages are not lawyer-reviewed.

5.3 UX, language, and access

Limitation	What that means
English only	No i18n; non-English questions will usually fall through to the fallback.
500 character input	Long support tickets do not belong here.
8-turn history cap	Older context is dropped on the server.
Thread not persisted	Reload or a new device starts from the welcome message.

No voice / no attachments / no markdown rendering beyond line breaks	Text only.
No streaming tokens	The visitor waits for the full answer (“Thinking…”).
Mobile overlap	The FAB can sit on top of other sticky UI on a different site unless z-index and offsets are retuned.
Tailwind + NexusQ tokens	A host without those classes needs a style map.

5.4 Operations and security

Limitation	What that means
No rate limiting	A bot can POST repeatedly. Fine for low marketing traffic; not hardened.
No unanswered-question log	You cannot see which asks failed unless you add logging later (avoid storing PII).
Public endpoint	Anyone can call `/api/help-chat`. Leakage risk is the knowledge file (and optional LLM traffic if a key is set).
No automated tests	The repo has no test script; behaviour is checked manually.

5.5 What this chatbox will never do in v1

- Answer general world knowledge.
- Place AuditionQ auditions or manage a performer’s account.
- Send email by itself (partner mail is a different form).
- Remember the visitor after refresh.
- Speak, take files, or take payments.

6. AI chatbox — future plan

Not in the current NexusQ v1 freeze unless Lead approves. Suggested order if the widget is copied to another live site:

1. **Rewrite knowledge for that product** (mandatory before go-live).
2. **Unanswered-question log** (no PII) to grow the knowledge file.
3. **Rate limit** `POST /api/help-chat` per IP.
4. **Handoff chip** into the partner/support form, prefilled from the last question.
5. **Persist thread** in `sessionStorage`.
6. **Generate answers from live page copy** so the file cannot drift.
7. **Optional grounded LLM** — already supported if `GROQ_API_KEY` or `OPENAI_API_KEY` is set; still facts-only. Not required for v1.
8. Streaming UI, i18n, or an admin FAQ CMS only if Lead expands scope.

7. How to implement the video box on another live website

> **Porting rule:** copy the component + CSS, then change four things: destination URL, labels, local MP4, poster image. Do not keep AuditionQ copy if the host site is a different product.

7.1 Files to copy

1. Component (TypeScript React, client component).
2. CSS block `.nq-casting-monitor` through the blink keyframes (Appendix A).
3. A local MP4 under the other site's static folder (example: `public/videos/product-bts.mp4`).
4. A poster still (WebP or JPEG).

7.2 Values to replace

NexusQ value	Change on the other site
`AUDITIONQ_URL`	That site's live URL or a specific in-app route
Label "Live audition"	Product-accurate status line
Badge "AuditionQ"	Host product name
Caption "Visit AuditionQ"	"Visit {product}" / "Open app"
`aria-label`	Match the destination
`VIDEO_SOURCES[0]`	Hosted MP4 on the same origin
CDN fallbacks	Remove or replace with licensed clips for that brand
`POSTER`	Product still from that site

7.3 If the other site is not Next.js

- Drop `"use client"`.
- Replace `useReducedMotion` from Framer Motion with `window.matchMedia("(prefers-reduced-motion: reduce)").matches`.
- Keep the same JSX structure (anchor wrapping the bezel).
- Paste the CSS as-is; class names do not require Tailwind.

7.4 Hero wiring (current NexusQ)

```
import CastingMonitor from "./CastingMonitor";

{/* Desktop — beside hero copy */}
<div className="hidden lg:flex lg:justify-end">
  <CastingMonitor />
</div>
```

```
{/* Mobile — under hero copy */}
<div className="relative z-20 mt-8 flex justify-center lg:hidden">
  <CastingMonitor />
</div>
```

7.5 Asset checklist

- MP4: H.264, muted-friendly, short loop (10–30s), keep under a few MB.
- Serve from the same origin so autoplay is reliable.
- Poster should match the first frame so load flash is small.
- iOS autoplay requires `muted` + `playsInline`.

7.6 Video box future plan (port phase 2)

1. Product-owned footage only — drop Mixkit/Pexels fallbacks.
2. Click analytics (destination, device, page).
3. Pause when the monitor leaves the viewport.
4. Optional same-tab vs new-tab.
5. Captions / still alt text.
6. Deep links (e.g. talent signup vs casting login) if the product wants two CTAs.

8. How to implement the AI chatbox on another live website

> **Do not paste the NexusQ knowledge file unchanged onto AuditionQ (or any other product).** Rewrite `knowledge`, `SITE_TERMS`, welcome copy, and suggestion chips so the bot only knows that product's public facts.

8.1 Recommended port steps

1. Copy `SiteHelpChat.tsx`, `route.ts`, `site-help.ts`, `site-help-paraphrase.ts`.
2. Mount the chat once in the root layout so it appears on all pages.
3. Rename "NexusQ Assistant" → "{Product} Assistant".
4. Replace every knowledge `answer` with that site's published copy only.
5. Replace `SITE_TERMS` with that product's names and routes.
6. Point links at that site's routes.
7. Keep the off-topic refuse behaviour and the honesty rules.
8. If the host is not Next.js: keep the engine as a plain TS/JS module; expose `POST /api/help-chat` with the same JSON contract (section 4.4).
9. Map Tailwind `nq-*` classes to the host design tokens.
10. If you cannot use Next `Link`, use ordinary `<a href>`.

8.2 Required CSS variables

```
--nq-bg
--nq-surface
--nq-surface-elevated
--nq-text
--nq-muted
--nq-border
--nq-accent-soft
```

8.3 Layout mount (current NexusQ)

```
import SiteHelpChat from "./components/SiteHelpChat";

export default function RootLayout({ children }) {
  return (
    <html lang="en">
      <body>
        {children}
        <SiteHelpChat />
      </body>
    </html>
  );
}
```

8.4 npm packages used by the chat UI

- `framer-motion` — panel enter/exit

- `lucide-react` — MessageCircle, Send, Sparkles, X
- Next.js App Router — `Link` and `app/api/help-chat/route.ts`

The retrieval engine itself has **zero** npm dependencies.

8.5 Suggested build order on the other site

1. Video box with first-party MP4 + click-through.
2. Help chat UI + small knowledge (who we are, how to sign in, how to contact).
3. Expand knowledge from that site's real pages only.
4. QA: off-topic refuse, live vs vision honesty, mobile FAB vs sticky bars.
5. Then analytics / optional grounded LLM / captions as a second phase.

9. Approval / port checklist

#	Check
1	Lead approves copying these two widgets to the other live website
2	Video destination URL, labels, and badge rewritten for that product
3	First-party video + poster licensed and hosted on that origin
4	Click opens the correct live product (desktop + mobile)
5	Reduced-motion users still get a working video-box link
6	Chat knowledge rewritten; NexusQ-only facts removed
7	Chat spec in this PDF is implemented (website-only, refuse general, 500-char cap)
8	Off-topic questions are refused; no invented metrics or fake live products
9	Help button does not cover primary CTAs on small screens
10	No new env secrets unless an LLM phase is approved

Appendix A — video box entire current code

A.1 app/components/CastingMonitor.tsx

```
"use client";

import { useReducedMotion } from "framer-motion";
import { useState } from "react";

/**
 * TEST ONLY — casting-room monitor with real behind-the-scenes footage.
 * Drop your own clip at public/videos/casting-bts.mp4 to override CDN sources.
 */

const VIDEO_SOURCES = [
  "/videos/casting-bts.mp4",
  // Mixkit — studio commercial shoot, BTS (free license)
  "https://assets.mixkit.co/videos/44062/44062-720.mp4",
  // Mixkit — backstage of filming (720p)
  "https://assets.mixkit.co/videos/13241/13241-720.mp4",
  // Pexels — indoor video shoot / lighting setup (BTS)
  "https://videos.pexels.com/video-files/33684356/33684356-sd_640_360_30fps.mp4",
] as const;

const POSTER = "/scenes/casting-audition.webp";
const AUDITIONQ_URL = "https://www.auditionq.com/";

export default function CastingMonitor() {
  const prefersReducedMotion = useReducedMotion();
  const [sourceIndex, setSourceIndex] = useState(0);
  const [usePoster, setUsePoster] = useState(false);

  const showVideo = !prefersReducedMotion && !usePoster;
  const source = VIDEO_SOURCES[sourceIndex];

  return (
    <a
      href={AUDITIONQ_URL}
      target="_blank"
      rel="noopener noreferrer"
      className="nq-casting-monitor"
      aria-label="Visit AuditionQ"
    >
      <div className="nq-casting-monitor-bezel">
        <div className="nq-casting-monitor-bar">
          <span className="nq-casting-monitor-rec" />
          <span className="nq-casting-monitor-label">Live audition</span>
          <span className="nq-casting-monitor-badge">AuditionQ</span>
        </div>
        <div className="nq-casting-monitor-screen">
          {showVideo ? (
            <video
              key={source}
              className="nq-casting-monitor-video"
              src={source}
              poster={POSTER}
              autoPlay
              muted
              loop
              playsInline
              preload="metadata"
              onError={() => {
                if (sourceIndex < VIDEO_SOURCES.length - 1) {
                  setSourceIndex((index) => index + 1);
                } else {
                  setUsePoster(true);
                }
              }}
            />
          ) : (
            <img
              src={POSTER}
              alt=""
```

```
              decoding="async"
              loading="lazy"
              draggable={false}
              className="nq-casting-monitor-video"
            />
          )}
          <div className="nq-casting-monitor-scan" />
        </div>
        <div className="nq-casting-monitor-caption">Visit AuditionQ</div>
      </div>
    </a>
  );
}
```

A.2 Monitor CSS from `app/globals.css`

```
.nq-casting-monitor {
  flex-shrink: 0;
  display: block;
  text-decoration: none;
  color: inherit;
  cursor: pointer;
}

.nq-casting-monitor-bezel {
  width: min(100%, 340px);
  overflow: hidden;
  border: 1px solid rgba(56, 189, 248, 0.22);
  border-radius: 16px;
  background: linear-gradient(165deg, #141820 0%, #0c0e14 100%);
  box-shadow:
    0 20px 50px rgba(0, 0, 0, 0.45),
    0 0 0 1px rgba(255, 255, 255, 0.06) inset,
    0 0 32px rgba(34, 211, 238, 0.08);
  color: #e8eef4;
  transition:
    border-color 0.25s var(--nq-ease),
    box-shadow 0.25s var(--nq-ease),
    transform 0.25s var(--nq-ease);
}

.nq-casting-monitor:hover .nq-casting-monitor-bezel,
.nq-casting-monitor:focus-visible .nq-casting-monitor-bezel {
  border-color: rgba(34, 211, 238, 0.55);
  box-shadow:
    0 22px 56px rgba(0, 0, 0, 0.5),
    0 0 0 1px rgba(255, 255, 255, 0.08) inset,
    0 0 42px rgba(34, 211, 238, 0.22);
  transform: translateY(-2px);
}

.nq-casting-monitor:focus-visible {
  outline: none;
}

@media (min-width: 1024px) {
  .nq-casting-monitor-bezel {
    width: 360px;
    border-radius: 18px;
  }
}

@media (min-width: 1280px) {
  .nq-casting-monitor-bezel {
    width: 400px;
  }
}

.nq-casting-monitor-bar {
  display: flex;
  align-items: center;
  gap: 0.55rem;
  padding: 0.55rem 0.85rem 0.5rem;
  font-size: 0.7rem;
  font-weight: 600;
  letter-spacing: 0.1em;
```

```
  border-bottom: 1px solid rgba(255, 255, 255, 0.06);
}

.nq-casting-monitor-rec {
  width: 0.5rem;
  height: 0.5rem;
  border-radius: 999px;
  background: #ef4444;
  box-shadow: 0 0 10px rgba(239, 68, 68, 0.75);
  flex-shrink: 0;
}

.nq-casting-monitor-label {
  text-transform: uppercase;
  letter-spacing: 0.14em;
  color: #cbd5e1;
}

.nq-casting-monitor-badge {
  margin-left: auto;
  padding: 0.15rem 0.5rem;
  border-radius: 999px;
  font-size: 0.62rem;
  font-weight: 700;
  letter-spacing: 0.08em;
  color: #67e8f9;
  background: rgba(34, 211, 238, 0.12);
  border: 1px solid rgba(34, 211, 238, 0.28);
}

.nq-casting-monitor-screen {
  position: relative;
  height: 190px;
  overflow: hidden;
  background: #080a0e;
}

@media (min-width: 1024px) {
  .nq-casting-monitor-screen {
    height: 210px;
  }
}

@media (min-width: 1280px) {
  .nq-casting-monitor-screen {
    height: 228px;
  }
}

.nq-casting-monitor-screen img,
.nq-casting-monitor-video {
  position: absolute;
  inset: 0;
  width: 100%;
  height: 100%;
  object-fit: cover;
  object-position: center;
  pointer-events: none;
}

.nq-casting-monitor-caption {
  padding: 0.5rem 0.85rem 0.65rem;
  font-size: 0.65rem;
  letter-spacing: 0.12em;
  text-transform: uppercase;
  color: #94a3b8;
}

.nq-casting-monitor-scan {
  position: absolute;
  inset: 0;
  z-index: 2;
  pointer-events: none;
  background: repeating-linear-gradient(
    to bottom,
    transparent 0,
    transparent 2px,
    rgba(0, 0, 0, 0.1) 3px
  );
```

```
  mix-blend-mode: multiply;
}

@media (prefers-reduced-motion: no-preference) {
  .nq-casting-monitor-rec {
    animation: nq-casting-monitor-blink 1.8s ease-in-out infinite;
  }

  .nq-casting-monitor-a {
    animation: nq-casting-monitor-ken 16s ease-in-out infinite,
      nq-casting-monitor-fade-a 12s ease-in-out infinite;
  }

  .nq-casting-monitor-b {
    animation: nq-casting-monitor-ken-rev 16s ease-in-out infinite,
      nq-casting-monitor-fade-b 12s ease-in-out infinite;
  }
}

@keyframes nq-casting-monitor-blink {
  0%,
  100% {
    opacity: 1;
  }
  50% {
    opacity: 0.35;
  }
}

@keyframes nq-casting-monitor-ken {
  0%,
  100% {
    transform: scale(1.05) translate3d(0, 0, 0);
  }
  50% {
    transform: scale(1.14) translate3d(-3%, -2%, 0);
  }
}

@keyframes nq-casting-monitor-ken-rev {
  0%,
  100% {
    transform: scale(1.12) translate3d(-2%, 0, 0);
  }
  50% {
    transform: scale(1.04) translate3d(2%, -2%, 0);
  }
}

@keyframes nq-casting-monitor-fade-a {
  0%,
  42%,
  100% {
    opacity: 1;
  }
  50%,
  92% {
    opacity: 0;
  }
}

@keyframes nq-casting-monitor-fade-b {
  0%,
  42%,
  100% {
    opacity: 0;
  }
  50%,
  92% {
    opacity: 1;
  }
}
```

Appendix B — AI chatbox entire current code

B.1 app/components/SiteHelpChat.tsx

```
"use client";

import { FormEvent, useEffect, useRef, useState } from "react";
import Link from "next/link";
import { AnimatePresence, motion } from "framer-motion";
import { MessageCircle, Send, Sparkles, X } from "lucide-react";

type HelpLink = {
  label: string;
  href: string;
  external?: boolean;
};

type ChatMessage = {
  id: string;
  role: "user" | "assistant";
  content: string;
  link?: HelpLink;
  suggestions?: string[];
};

const STARTER: ChatMessage = {
  id: "welcome",
  role: "assistant",
  content:
    "Hi — I can help with this NexusQ Global website only: who we are, which products are live or still vision, AuditionQ,
partnering, and sign-in. What would you like to know?",
  suggestions: [
    "What is NexusQ Global?",
    "Is AuditionQ live?",
    "How do I partner with you?",
    "How do I sign in?",
  ],
};

function nextId() {
  return `${Date.now()}-${Math.random().toString(36).slice(2, 8)}`;
}

export default function SiteHelpChat() {
  const [open, setOpen] = useState(false);
  const [input, setInput] = useState("");
  const [loading, setLoading] = useState(false);
  const [messages, setMessages] = useState<ChatMessage[]>([STARTER]);
  const listRef = useRef<HTMLDivElement>(null);
  const inputRef = useRef<HTMLInputElement>(null);

  useEffect(() => {
    if (!open) return;
    const onKey = (e: KeyboardEvent) => {
      if (e.key === "Escape") setOpen(false);
    };
    document.addEventListener("keydown", onKey);
    const id = window.setTimeout(() => inputRef.current?.focus(), 80);
    return () => {
      document.removeEventListener("keydown", onKey);
      window.clearTimeout(id);
    };
  }, [open]);

  useEffect(() => {
    const el = listRef.current;
    if (!el) return;
    el.scrollTop = el.scrollHeight;
  }, [messages, loading, open]);

  async function send(text: string) {
    const message = text.trim();
    if (!message || loading) return;
```

```
  const userMessage: ChatMessage = {
    id: nextId(),
    role: "user",
    content: message,
  };

  const nextMessages = [...messages, userMessage];
  setMessages(nextMessages);
  setInput("");
  setLoading(true);

  try {
    const history = nextMessages
      .filter((m) => m.id !== "welcome")
      .slice(0, -1)
      .map((m) => ({ role: m.role, content: m.content }));

    const res = await fetch("/api/help-chat", {
      method: "POST",
      headers: { "Content-Type": "application/json" },
      body: JSON.stringify({ message, history }),
    });
    const data = await res.json();

    if (!res.ok) {
      setMessages((prev) => [
        ...prev,
        {
          id: nextId(),
          role: "assistant",
          content: data.error || "Something went wrong. Please try again.",
          suggestions: STARTER.suggestions,
        },
      ]);
      return;
    }

    setMessages((prev) => [
      ...prev,
      {
        id: nextId(),
        role: "assistant",
        content: data.answer,
        link: data.link,
        suggestions: data.suggestions,
      },
    ]);
  } catch {
    setMessages((prev) => [
      ...prev,
      {
        id: nextId(),
        role: "assistant",
        content: "Network error. Please try again.",
        suggestions: STARTER.suggestions,
      },
    ]);
  } finally {
    setLoading(false);
  }
}

function onSubmit(e: FormEvent) {
  e.preventDefault();
  void send(input);
}

const lastAssistant = [...messages].reverse().find((m) => m.role === "assistant");

return (
  <div className="fixed bottom-5 right-4 z-[80] sm:bottom-6 sm:right-6">
    <AnimatePresence>
      {open ? (
        <motion.section
          key="panel"
          initial={{ opacity: 0, y: 16, scale: 0.96 }}
          animate={{ opacity: 1, y: 0, scale: 1 }}
          exit={{ opacity: 0, y: 16, scale: 0.96 }}
```

```
            transition={{ duration: 0.18 }}
            aria-label="NexusQ site help chat"
            className="mb-3 flex h-[min(32rem,calc(100dvh-5.5rem))] w-[min(22.5rem,calc(100vw-2rem))] flex-col overflow-hidden
rounded-2xl border border-nq-border bg-nq-surface shadow-[0_18px_50px_rgba(0,0,0,0.45)]"
          >
            <header className="flex items-center justify-between gap-3 border-b border-nq-border bg-nq-surface-elevated px-4 py-
3">
              <div className="flex min-w-0 items-center gap-2.5">
                <span className="flex h-8 w-8 items-center justify-center rounded-full bg-cyan-500/15 text-cyan-400">
                  <Sparkles className="h-4 w-4" aria-hidden />
                </span>
                <div className="min-w-0">
                  <p className="text-sm font-semibold text-nq-text">NexusQ Assistant</p>
                  <p className="text-[11px] text-nq-muted">Website help only</p>
                </div>
              </div>
              <button
                type="button"
                onClick={() => setOpen(false)}
                className="inline-flex h-8 w-8 items-center justify-center rounded-lg text-nq-muted hover:bg-nq-accent-soft
hover:text-nq-text"
                aria-label="Close help chat"
              >
                <X className="h-4 w-4" />
              </button>
            </header>

            <div
              ref={listRef}
              className="min-h-0 flex-1 space-y-3 overflow-y-auto px-3 py-3"
            >
              {messages.map((msg) => (
                <div
                  key={msg.id}
                  className={
                    msg.role === "user" ? "flex justify-end" : "flex justify-start"
                  }
                >
                  <div
                    className={
                      msg.role === "user"
                        ? "max-w-[85%] rounded-2xl rounded-br-md bg-cyan-500 px-3.5 py-2 text-sm leading-relaxed text-slate-950"
                        : "max-w-[90%] rounded-2xl rounded-bl-md border border-nq-border bg-nq-surface-elevated px-3.5 py-2 text-
sm leading-relaxed text-nq-text"
                    }
                  >
                    <p className="whitespace-pre-wrap">{msg.content}</p>
                    {msg.role === "assistant" && msg.link ? (
                      msg.link.external ? (
                        <a
                          href={msg.link.href}
                          target="_blank"
                          rel="noopener noreferrer"
                          className="mt-2 inline-flex text-xs font-medium text-cyan-400 hover:underline"
                        >
                          {msg.link.label} →
                        </a>
                      ) : (
                        <Link
                          href={msg.link.href}
                          className="mt-2 inline-flex text-xs font-medium text-cyan-400 hover:underline"
                          onClick={() => setOpen(false)}
                        >
                          {msg.link.label} →
                        </Link>
                      )
                    ) : null}
                  </div>
                </div>
              ))}

              {loading ? (
                <div className="flex justify-start">
                  <div className="rounded-2xl border border-nq-border bg-nq-surface-elevated px-3.5 py-2 text-xs text-nq-muted">
                    Thinking…
                  </div>
                </div>
              ) : null}
            </div>
```

```
            {!loading && lastAssistant?.suggestions?.length ? (
              <div className="flex flex-wrap gap-1.5 border-t border-nq-border px-3 py-2">
                {lastAssistant.suggestions.map((suggestion) => (
                  <button
                    key={suggestion}
                    type="button"
                    onClick={() => void send(suggestion)}
                    className="rounded-full border border-nq-border bg-nq-surface-elevated px-2.5 py-1 text-[11px] text-nq-muted
hover:border-cyan-400/40 hover:text-nq-text"
                  >
                    {suggestion}
                  </button>
                ))}
              </div>
            ) : null}

            <form
              onSubmit={onSubmit}
              className="flex items-center gap-2 border-t border-nq-border px-3 py-2.5"
            >
              <input
                ref={inputRef}
                value={input}
                onChange={(e) => setInput(e.target.value)}
                maxLength={500}
                placeholder="Ask about this website…"
                aria-label="Ask about this website"
                className="min-w-0 flex-1 rounded-xl border border-nq-border bg-nq-bg px-3 py-2 text-sm text-nq-text outline-none
placeholder:text-nq-muted focus:border-cyan-400/50"
              />
              <button
                type="submit"
                disabled={loading || !input.trim()}
                className="inline-flex h-9 w-9 shrink-0 items-center justify-center rounded-xl bg-cyan-500 text-slate-950
disabled:opacity-40"
                aria-label="Send message"
              >
                <Send className="h-4 w-4" />
              </button>
            </form>
          </motion.section>
        ) : null}
      </AnimatePresence>

      {!open ? (
        <button
          type="button"
          onClick={() => setOpen(true)}
          aria-expanded={false}
          aria-label="Open NexusQ help chat"
          className="ml-auto flex items-center gap-2 rounded-full bg-cyan-500 px-3 py-2.5 text-xs font-semibold text-slate-950
shadow-lg transition hover:bg-cyan-400 active:scale-95 sm:px-4 sm:py-3 sm:text-sm"
        >
          <span className="relative flex h-2 w-2">
            <span className="absolute inline-flex h-full w-full animate-ping rounded-full bg-emerald-400 opacity-75" />
            <span className="relative inline-flex h-2 w-2 rounded-full bg-emerald-500" />
          </span>
          <MessageCircle className="h-4 w-4" aria-hidden />
          Help
        </button>
      ) : null}
    </div>
  );
}
```

B.2 `app/api/help-chat/route.ts`

```
import { NextResponse } from "next/server";
import {
  composeSiteHelp,
  retrieveSiteHelp,
  type ChatTurn,
} from "@/lib/site-help";
import { paraphraseSiteHelp } from "@/lib/site-help-paraphrase";

const MAX_MESSAGE = 500;
const MAX_HISTORY = 8;

function asHistory(value: unknown): ChatTurn[] {
  if (!Array.isArray(value)) return [];
  return value
    .slice(-MAX_HISTORY)
    .flatMap((item) => {
      if (!item || typeof item !== "object") return [];
      const role = "role" in item ? String(item.role) : "";
      const content = "content" in item ? String(item.content) : "";
      if ((role !== "user" && role !== "assistant") || !content.trim()) return [];
      return [
        {
          role,
          content: content.trim().slice(0, MAX_MESSAGE),
        } satisfies ChatTurn,
      ];
    });
}

export async function POST(req: Request) {
  try {
    const body = await req.json();
    const message = String(body.message ?? "").trim();

    if (!message) {
      return NextResponse.json(
        { error: "Please type a question about this website." },
        { status: 400 }
      );
    }

    if (message.length > MAX_MESSAGE) {
      return NextResponse.json(
        { error: "Please keep questions under 500 characters." },
        { status: 400 }
      );
    }

    const history = asHistory(body.history);
    const retrieved = retrieveSiteHelp(message, history);
    const composed = composeSiteHelp(retrieved, message, history);
    const reply = await paraphraseSiteHelp(message, history, retrieved, composed);
    return NextResponse.json(reply);
  } catch {
    return NextResponse.json(
      { error: "Could not answer right now. Please try again." },
      { status: 500 }
    );
  }
}
```

B.3 `lib/site-help.ts` (facts + composer)

```
export type ChatRole = "user" | "assistant";

export type ChatTurn = {
  role: ChatRole;
  content: string;
};

export type HelpLink = {
```

```
  label: string;
  href: string;
  external?: boolean;
};

export type HelpReply = {
  answer: string;
  link?: HelpLink;
  suggestions: string[];
};

type Intent = "what" | "how" | "where" | "why" | "yesno" | "list" | "general";

type Fact = {
  text: string;
  intents?: Intent[];
};

type KnowledgeEntry = {
  id: string;
  title: string;
  keywords: string[];
  facts: Fact[];
  link?: HelpLink;
  suggestions: string[];
};

export type RetrievedHelp = {
  kind: "empty" | "greeting" | "thanks" | "off_topic" | "unknown" | "answer";
  intent: Intent;
  query: string;
  facts: string[];
  link?: HelpLink;
  suggestions: string[];
  lead?: string;
};

const SITE_TERMS = [
  "nexusq",
  "nexus q",
  "auditionq",
  "audition q",
  "fursure",
  "fur sure",
  "rideq",
  "ride q",
  "caringminds",
  "caring minds",
  "onakkodi",
  "onak kodi",
  "future ai",
  "partner",
  "partnership",
  "collaborate",
  "login",
  "sign in",
  "sign up",
  "account",
  "privacy",
  "terms",
  "website",
  "this site",
  "this page",
  "homepage",
  "product",
  "products",
  "ecosystem",
  "flagship",
  "vision",
  "exploration",
  "navbar",
  "navigation",
  "guide me",
  "tour",
  "theme",
  "dark mode",
  "light mode",
  "contact",
  "email",
```

```
  "inquiry",
  "enquiry",
  "admin@",
  "auditionq.com",
  "video box",
  "monitor",
];

const OFF_TOPIC = [
  "weather",
  "recipe",
  "recipes",
  "joke",
  "jokes",
  "homework",
  "capital of",
  "who won",
  "stock price",
  "bitcoin",
  "crypto",
  "write python",
  "write javascript",
  "write code",
  "translate this",
  "poem",
  "essay",
  "news today",
  "sports score",
];

const DEFAULT_SUGGESTIONS = [
  "What is NexusQ Global?",
  "Is AuditionQ live?",
  "How do I partner with you?",
  "How do I sign in?",
];

const GREETING = /^(hi|hello|hey|yo|hiya|good (morning|afternoon|evening)|howdy)\b/i;
const THANKS =
  /^(thanks|thank you|thx|cheers|great|awesome|perfect|got it|ok|okay|cool|nice)\b/i;

const knowledge: KnowledgeEntry[] = [
  {
    id: "what-is-nexusq",
    title: "What is NexusQ Global",
    keywords: [
      "nexusq",
      "nexus q",
      "who are you",
      "what is this",
      "this website",
      "this site",
      "company",
      "parent company",
      "about",
      "what do you do",
      "who is nexusq",
      "overview",
    ],
    facts: [
      {
        text: "NexusQ Global is a product-first parent company that designs and ships focused digital platforms.",
        intents: ["what", "general"],
      },
      {
        text: "This website is the company home: it introduces the ecosystem and how to partner with the team.",
        intents: ["what", "how", "general"],
      },
      {
        text: "NexusQ is not a single-app company — AuditionQ is the live flagship, and other named platforms stay labelled until
they ship.",
        intents: ["what", "list", "yesno"],
      },
    ],
    link: { label: "Go to Home", href: "/#home" },
    suggestions: [
      "What products are in the ecosystem?",
      "Is AuditionQ live?",
      "How do I partner with you?",
```

```
    ],
  },
  {
    id: "how-to-use-site",
    title: "How to use this website",
    keywords: [
      "how to use",
      "how does this site",
      "navigate",
      "where do i start",
      "help me around",
      "help",
      "guide",
      "tour",
      "sections",
      "what can i do here",
      "show me around",
    ],
    facts: [
      {
        text: "Use the top navigation to jump around the homepage.",
        intents: ["how", "where", "general"],
      },
      {
        text: "Home introduces NexusQ. Platforms lists every product with Live, Vision, or Exploration labels.",
        intents: ["how", "list", "what"],
      },
      {
        text: "You can click the homepage casting video box to open AuditionQ, or use the AuditionQ section CTA.",
        intents: ["how", "where"],
      },
      {
        text: "Sign In is in the navbar. Theme toggle and Guide me (spotlight tour) are in the menu as well.",
        intents: ["how", "where"],
      },
    ],
    link: { label: "Explore platforms", href: "/#platforms" },
    suggestions: [
      "What is the difference between Live and Vision?",
      "How do I contact you?",
      "Where is AuditionQ?",
    ],
  },
  {
    id: "auditionq",
    title: "AuditionQ",
    keywords: [
      "auditionq",
      "audition q",
      "flagship",
      "live product",
      "talent",
      "casting",
      "audition",
      "interview",
      "auditionq.com",
      "video box",
      "monitor",
      "casting room",
      "open",
      "visit",
      "click",
    ],
    facts: [
      {
        text: "AuditionQ is NexusQ Global’s live flagship product.",
        intents: ["what", "yesno", "general"],
      },
      {
        text: "It is a modern, AI-powered platform that connects talent with opportunities through an audition and interview
experience.",
        intents: ["what", "general"],
      },
      {
        text: "AuditionQ is available today at https://www.auditionq.com/.",
        intents: ["where", "how", "yesno"],
      },
      {
        text: "On this homepage, click the casting video box (Live audition / AuditionQ) to open that site.",
```

```
        intents: ["how", "where"],
      },
      {
        text: "AuditionQ is a distinct product; NexusQ Global remains the parent company. It is the only platform on this site
marked Live.",
        intents: ["yesno", "what", "list"],
      },
    ],
    link: {
      label: "Visit AuditionQ",
      href: "https://www.auditionq.com/",
      external: true,
    },
    suggestions: [
      "Are any other products live?",
      "How do I partner with NexusQ?",
      "What is Future AI?",
    ],
  },
  {
    id: "ecosystem",
    title: "Product ecosystem",
    keywords: [
      "ecosystem",
      "platforms",
      "products",
      "suite",
      "all products",
      "what products",
      "what do you build",
      "product list",
    ],
    facts: [
      {
        text: "The ecosystem is AuditionQ (Live), FurSure (Vision), RideQ (Vision), CaringMinds (Vision), Onakkodi (Vision), and
Future AI (Exploration).",
        intents: ["list", "what", "general"],
      },
      {
        text: "Only AuditionQ is available today.",
        intents: ["yesno", "list"],
      },
      {
        text: "Vision and exploration products have no download, store, or “launch app” buttons until they ship.",
        intents: ["how", "yesno", "what"],
      },
    ],
    link: { label: "See platforms", href: "/#platforms" },
    suggestions: [
      "Tell me about FurSure",
      "Tell me about RideQ",
      "Is AuditionQ live?",
    ],
  },
  {
    id: "live-vs-vision",
    title: "Live, vision, and exploration",
    keywords: [
      "live",
      "vision",
      "exploration",
      "status",
      "launched",
      "available",
      "not launched",
      "coming soon",
      "download",
      "difference between",
      "labels",
    ],
    facts: [
      {
        text: "Live means the product is publicly available now — that is only AuditionQ.",
        intents: ["what", "yesno", "list"],
      },
      {
        text: "Vision means a named direction or early concept that is not launched and is not offered as a service.",
        intents: ["what", "yesno"],
      },
```

```
      {
        text: "Exploration (Future AI) is research and experimentation, not a shipped product.",
        intents: ["what", "yesno"],
      },
      {
        text: "We do not invent launch dates, store links, or fake metrics.",
        intents: ["why", "what"],
      },
    ],
    suggestions: [
      "What is AuditionQ?",
      "Which products are vision?",
      "How do I partner?",
    ],
  },
  {
    id: "fursure",
    title: "FurSure",
    keywords: ["fursure", "fur sure", "pet", "paw"],
    facts: [
      {
        text: "FurSure is a vision product in the NexusQ ecosystem.",
        intents: ["what", "yesno", "general"],
      },
      {
        text: "Details will be shared when it moves beyond exploration. It is not launched and cannot be downloaded or used
yet.",
        intents: ["what", "yesno", "how"],
      },
    ],
    link: { label: "See platforms", href: "/#platforms" },
    suggestions: ["What else is in vision?", "Is AuditionQ live?"],
  },
  {
    id: "rideq",
    title: "RideQ",
    keywords: ["rideq", "ride q", "mobility", "car", "transport"],
    facts: [
      {
        text: "RideQ is a vision product focused on future mobility experiences.",
        intents: ["what", "general"],
      },
      {
        text: "It is in early concept and is not launched.",
        intents: ["yesno", "what"],
      },
    ],
    link: { label: "See platforms", href: "/#platforms" },
    suggestions: ["What is CaringMinds?", "How do I partner?"],
  },
  {
    id: "caringminds",
    title: "CaringMinds",
    keywords: ["caringminds", "caring minds", "wellbeing", "well being", "care", "health"],
    facts: [
      {
        text: "CaringMinds is a vision product exploring digital tools for wellbeing and care.",
        intents: ["what", "general"],
      },
      {
        text: "It is presented as direction, not a live service.",
        intents: ["yesno", "what"],
      },
    ],
    link: { label: "See platforms", href: "/#platforms" },
    suggestions: ["What is Onakkodi?", "Is AuditionQ live?"],
  },
  {
    id: "onakkodi",
    title: "Onakkodi",
    keywords: ["onakkodi", "onak kodi", "apparel", "fashion", "clothing"],
    facts: [
      {
        text: "Onakkodi is a vision product under the NexusQ umbrella.",
        intents: ["what", "general"],
      },
      {
        text: "Status is concept / early development — it is not available yet.",
        intents: ["yesno", "what"],
```

```
      },
    ],
    link: { label: "See platforms", href: "/#platforms" },
    suggestions: ["What is Future AI?", "How do I partner?"],
  },
  {
    id: "future-ai",
    title: "Future AI",
    keywords: [
      "future ai",
      "ai",
      "ai agents",
      "artificial intelligence",
      "innovation",
      "experiment",
      "next generation",
    ],
    facts: [
      {
        text: "Future AI is ongoing exploration into AI capabilities and future platforms — experimentation only, not a shipped
product.",
        intents: ["what", "yesno", "general"],
      },
      {
        text: "The Future section also mentions AI agents, global expansion, and future platforms as directions, introduced only
when ready.",
        intents: ["what", "list", "how"],
      },
    ],
    link: { label: "See Future", href: "/#future" },
    suggestions: ["What is live today?", "How do I partner?"],
  },
  {
    id: "partner",
    title: "Partner with NexusQ",
    keywords: [
      "partner",
      "partnership",
      "collaborate",
      "collaboration",
      "client",
      "business",
      "investment",
      "contact",
      "inquiry",
      "enquiry",
      "work with",
      "reach out",
      "get in touch",
      "start a conversation",
    ],
    facts: [
      {
        text: "NexusQ welcomes partnerships, client work, collaboration, investment/business conversations, and other legitimate
inquiries.",
        intents: ["what", "general"],
      },
      {
        text: "Open /partner, then send your name, email, optional company, interest type, and a message. The team reads every
submission.",
        intents: ["how", "where"],
      },
      {
        text: "You can also email admin@auditionq.com.",
        intents: ["how", "where"],
      },
    ],
    link: { label: "Open partner form", href: "/partner" },
    suggestions: [
      "What email can I use?",
      "Is AuditionQ live?",
      "What is NexusQ Global?",
    ],
  },
  {
    id: "email",
    title: "Contact email",
    keywords: ["email", "mail", "admin@", "write to you", "phone", "address"],
    facts: [
```

```
      {
        text: "The contact address published on this site is admin@auditionq.com.",
        intents: ["where", "how", "what"],
      },
      {
        text: "For partnerships, the Partner form at /partner is the preferred path.",
        intents: ["how", "where"],
      },
      {
        text: "We do not publish a street address or phone number on this website.",
        intents: ["where", "what", "yesno"],
      },
    ],
    link: { label: "Open partner form", href: "/partner" },
    suggestions: ["How does the partner form work?", "Where is Privacy?"],
  },
  {
    id: "login",
    title: "Sign in and sign up",
    keywords: [
      "login",
      "log in",
      "sign in",
      "sign up",
      "signup",
      "register",
      "account",
      "password",
      "create account",
      "sign out",
      "settings",
    ],
    facts: [
      {
        text: "Use Sign In in the navbar to open /login. You can create an account with name, email, and password, or sign in if
you already have one.",
        intents: ["how", "where"],
      },
      {
        text: "After you sign in, the navbar shows your name and Sign Out.",
        intents: ["how", "what"],
      },
      {
        text: "This account is for the NexusQ Global website — it is not an AuditionQ product login. There is no Settings page on
this site.",
        intents: ["what", "yesno"],
      },
    ],
    link: { label: "Go to Sign In", href: "/login" },
    suggestions: ["What data do you store?", "How do I partner?"],
  },
  {
    id: "privacy",
    title: "Privacy",
    keywords: ["privacy", "data", "personal information", "cookies", "gdpr"],
    facts: [
      {
        text: "The Privacy page is a placeholder pending final legal review — not lawyer-reviewed legal advice.",
        intents: ["what", "general"],
      },
      {
        text: "The partner form collects name, email, company, interest, and message. Accounts store name, email, and a hashed
password.",
        intents: ["what", "how"],
      },
      {
        text: "Privacy questions: admin@auditionq.com.",
        intents: ["how", "where"],
      },
    ],
    link: { label: "Read Privacy", href: "/privacy" },
    suggestions: ["Where are the Terms?", "How do I sign in?"],
  },
  {
    id: "terms",
    title: "Terms of use",
    keywords: ["terms", "legal", "conditions", "warranty"],
    facts: [
      {
```

```
        text: "The Terms page is a placeholder pending final legal review.",
        intents: ["what", "general"],
      },
      {
        text: "AuditionQ is described as live; other named platforms may be vision or exploration and are not offered as
available services unless explicitly stated.",
        intents: ["what", "yesno"],
      },
    ],
    link: { label: "Read Terms", href: "/terms" },
    suggestions: ["Where is Privacy?", "What is live today?"],
  },
  {
    id: "trust",
    title: "Trust and honesty",
    keywords: [
      "trust",
      "metrics",
      "testimonials",
      "customers",
      "proof",
      "honest",
      "hype",
    ],
    facts: [
      {
        text: "This website does not publish fabricated metrics, testimonials, or customer logos.",
        intents: ["what", "why", "yesno"],
      },
      {
        text: "Trust is meant to come from shipped work (AuditionQ is live) and clear Live vs Vision labels.",
        intents: ["what", "why"],
      },
    ],
    link: { label: "See Trust", href: "/#trust" },
    suggestions: ["Is AuditionQ live?", "How do I partner?"],
  },
  {
    id: "theme",
    title: "Light and dark theme",
    keywords: ["theme", "dark mode", "light mode", "appearance", "toggle"],
    facts: [
      {
        text: "The navbar includes a theme toggle for dark (default) and light mode.",
        intents: ["how", "where", "what"],
      },
      {
        text: "The same pages and product labels stay available in both themes.",
        intents: ["what", "yesno"],
      },
    ],
    suggestions: ["How do I navigate the site?", "How do I sign in?"],
  },
  {
    id: "vision-company",
    title: "Why NexusQ exists",
    keywords: ["why", "pillars", "craft", "mission", "about us"],
    facts: [
      {
        text: "NexusQ exists to build and steward an ecosystem of focused digital products — not a single app.",
        intents: ["why", "what"],
      },
      {
        text: "We ship what is real (AuditionQ is live) and keep vision products clearly labelled until they are ready.",
        intents: ["why", "what", "yesno"],
      },
    ],
    link: { label: "Read why we exist", href: "/#about" },
    suggestions: ["What products exist?", "How do I partner?"],
  },
];

function normalize(text: string) {
  return text.toLowerCase().replace(/[^a-z0-9@.#+\s]/g, " ").replace(/\s+/g, " ").trim();
}

function seedFrom(text: string, extra = 0) {
  let h = extra + 7;
  for (let i = 0; i < text.length; i++) h = (Math.imul(h, 31) + text.charCodeAt(i)) >>> 0;
```

```
  return h;
}

function pick<T>(items: readonly T[], seed: number): T {
  return items[seed % items.length];
}

function isOffTopic(query: string) {
  if (SITE_TERMS.some((term) => query.includes(term))) return false;
  return OFF_TOPIC.some((term) => query.includes(term));
}

function isSiteRelated(query: string) {
  return SITE_TERMS.some((term) => query.includes(term));
}

function detectIntent(query: string): Intent {
  if (/\b(how do i|how can i|how to|how does|steps?)\b/.test(query)) return "how";
  if (/\bwhere\b/.test(query)) return "where";
  if (/\bwhy\b/.test(query)) return "why";
  if (/\b(which|list|all product|ecosystem|platforms|what products)\b/.test(query)) {
    return "list";
  }
  if (
    /^(is|are|does|do|can|will)\b/.test(query) ||
    /\b(is it live|are they live|available yet|launched yet|not launched)\b/.test(query)
  ) {
    return "yesno";
  }
  if (/\b(what is|what's|who is|tell me about|explain)\b/.test(query)) return "what";
  return "general";
}

function scoreEntry(query: string, entry: KnowledgeEntry) {
  let score = 0;
  const qTokens = new Set(query.split(" ").filter((t) => t.length > 1));

  for (const keyword of entry.keywords) {
    if (query.includes(keyword)) {
      score += keyword.includes(" ") ? 8 : 5;
    } else {
      const kTokens = keyword.split(" ");
      const overlap = kTokens.filter((t) => qTokens.has(t)).length;
      if (overlap && overlap === kTokens.length) score += 3;
    }
  }

  for (const word of entry.title.toLowerCase().split(" ")) {
    if (word.length > 2 && qTokens.has(word)) score += 2;
  }

  return score;
}

function expandWithHistory(message: string, history: ChatTurn[]) {
  const trimmed = message.trim();
  const words = trimmed.split(/\s+/);
  const followUp =
    words.length <= 10 &&
    /^(and |what about|how about|tell me more|more|is it|is that|can i|where|how|why|that|this|them|those|it |the
|also)\b/i.test(
      trimmed
    );

  if (!followUp) return trimmed;

  const lastUser = [...history].reverse().find((t) => t.role === "user");
  const lastAssistant = [...history].reverse().find((t) => t.role === "assistant");
  return [lastUser?.content, lastAssistant?.content.slice(0, 180), trimmed]
    .filter(Boolean)
    .join(" ");
}

function alreadySaid(history: ChatTurn[]) {
  return history
    .filter((t) => t.role === "assistant")
    .map((t) => t.content.toLowerCase())
    .join("\n");
}
```

```
function yesNoLead(query: string, seed: number): string | undefined {
  const liveYes = /\b(auditionq|audition q)\b/.test(query) && /\b(live|available|launched|out yet)\b/.test(query);
  const liveAny = /\b(any other|other product|anything else).*(live|available)/.test(query);
  const visionName =
    /\b(fursure|fur sure|rideq|ride q|caringminds|caring minds|onakkodi|future ai)\b/.test(
      query
    ) && /\b(live|available|launched|out yet|ready)\b/.test(query);

  if (liveYes) {
    return pick(
      [
        "Yes — AuditionQ is live today.",
        "Yes. AuditionQ is the only live product on this site.",
        "Short answer: yes. AuditionQ is live.",
      ],
      seed
    );
  }
  if (liveAny) {
    return pick(
      [
        "No — only AuditionQ is live.",
        "Nothing else is live yet. AuditionQ is the one shipped product.",
        "Just AuditionQ for now. The others are vision or exploration.",
      ],
      seed
    );
  }
  if (visionName) {
    return pick(
      [
        "No — that one is not launched.",
        "Not yet. It is still labelled vision or exploration, not a live service.",
        "It is not available to use today.",
      ],
      seed
    );
  }
  return undefined;
}

function selectFacts(
  entries: KnowledgeEntry[],
  intent: Intent,
  said: string,
  limit: number
) {
  const picked: string[] = [];
  for (const entry of entries) {
    const unused = entry.facts.filter((f) => !said.includes(f.text.slice(0, 48).toLowerCase()));
    const tagged = unused.filter((f) => f.intents?.includes(intent));
    const fallback = unused.filter((f) => !tagged.includes(f));
    const pool = [...tagged, ...fallback].map((f) => f.text);
    for (const text of pool) {
      if (picked.includes(text)) continue;
      picked.push(text);
      if (picked.length >= limit) return picked;
    }
  }
  if (picked.length === 0) {
    for (const entry of entries) {
      for (const fact of entry.facts) {
        if (!picked.includes(fact.text)) picked.push(fact.text);
        if (picked.length >= limit) return picked;
      }
    }
  }
  return picked;
}

export function retrieveSiteHelp(
  rawMessage: string,
  history: ChatTurn[] = []
): RetrievedHelp {
  const message = rawMessage.trim();
  const seed = seedFrom(message, history.length);

  if (!message) {
```

```
    return {
      kind: "empty",
      intent: "general",
      query: "",
      facts: [],
      suggestions: DEFAULT_SUGGESTIONS,
    };
  }

  if (GREETING.test(message) && message.split(/\s+/).length <= 6) {
    return {
      kind: "greeting",
      intent: "general",
      query: normalize(message),
      facts: [],
      suggestions: DEFAULT_SUGGESTIONS,
    };
  }

  if (THANKS.test(message) && message.split(/\s+/).length <= 8) {
    return {
      kind: "thanks",
      intent: "general",
      query: normalize(message),
      facts: [],
      suggestions: DEFAULT_SUGGESTIONS,
    };
  }

  const query = normalize(expandWithHistory(message, history));
  const intent = detectIntent(query);

  if (isOffTopic(query) && !isSiteRelated(query)) {
    return {
      kind: "off_topic",
      intent,
      query,
      facts: [],
      suggestions: DEFAULT_SUGGESTIONS,
    };
  }

  const ranked = knowledge
    .map((entry) => ({ entry, score: scoreEntry(query, entry) }))
    .sort((a, b) => b.score - a.score);

  const best = ranked[0];
  const second = ranked[1];
  const minScore = isSiteRelated(query) ? 3 : 5;

  if (!best || best.score < minScore) {
    return {
      kind: "unknown",
      intent,
      query,
      facts: [],
      suggestions: DEFAULT_SUGGESTIONS,
    };
  }

  const chosen = [best.entry];
  if (
    second &&
    second.score >= 5 &&
    second.score >= best.score * 0.72 &&
    second.entry.id !== best.entry.id
  ) {
    chosen.push(second.entry);
  }

  const factLimit =
    intent === "yesno" ? (yesNoLead(query, seed) ? 1 : 2) : intent === "where" || intent === "how" ? 2 : 3;
  const facts = selectFacts(chosen, intent, alreadySaid(history), factLimit);

  if (facts.length === 0) {
    return {
      kind: "answer",
      intent,
      query,
```

```
      facts: [],
      link: best.entry.link ?? second?.entry.link,
      suggestions: best.entry.suggestions,
      lead: pick(
        [
          "I already shared what this site says on that. Ask a follow-up — partnering, sign-in, or another product.",
          "That’s the extent of the public notes on that topic. What else about NexusQ would you like?",
          "Covered from the site guide already. Try a related question from the chips below.",
        ],
        seed
      ),
    };
  }

  return {
    kind: "answer",
    intent,
    query,
    facts,
    link: best.entry.link ?? second?.entry.link,
    suggestions: best.entry.suggestions,
    lead: intent === "yesno" ? yesNoLead(query, seed) : undefined,
  };
}

function stitch(facts: string[]) {
  return facts.filter(Boolean).join(" ");
}

export function composeSiteHelp(retrieved: RetrievedHelp, rawMessage: string, history: ChatTurn[] = []): HelpReply {
  const seed = seedFrom(rawMessage.trim(), history.length);

  if (retrieved.kind === "empty") {
    return {
      answer: pick(
        [
          "Ask anything about NexusQ Global, our products, or how to use this website.",
          "I can help with this site — products, partnering, or sign-in. What do you want to know?",
          "Fire away: NexusQ, AuditionQ, vision products, partnering, or navigating the site.",
        ],
        seed
      ),
      suggestions: DEFAULT_SUGGESTIONS,
    };
  }

  if (retrieved.kind === "greeting") {
    return {
      answer: pick(
        [
          "Hi — I am the NexusQ site assistant. I only cover this website: who NexusQ is, which products are live or still
vision, AuditionQ, partnering, and sign-in. What would you like to know?",
          "Hello. I can guide you around NexusQ Global — not general topics. Ask about AuditionQ, the ecosystem, or how to get in
touch.",
          "Hey. I am here for questions about this NexusQ site. Try products, Live vs Vision, partnering, or signing in.",
        ],
        seed
      ),
      suggestions: DEFAULT_SUGGESTIONS,
    };
  }

  if (retrieved.kind === "thanks") {
    return {
      answer: pick(
        [
          "Glad that helped. Ask another website question whenever you want.",
          "You are welcome. I can also explain partnering, sign-in, or which products are still vision.",
          "Anytime. If you want the next step — visiting AuditionQ or the partner form — just say so.",
        ],
        seed
      ),
      suggestions: DEFAULT_SUGGESTIONS,
    };
  }

  if (retrieved.kind === "off_topic") {
    return {
      answer: pick(
```

```
        [
          "I only answer questions about the NexusQ Global website and products — not general topics. Try AuditionQ, vision
platforms, partnering, or sign-in.",
          "That is outside what I cover. Stick to this site: NexusQ, AuditionQ, partnering, or how the pages work.",
          "I cannot help with that. Ask me about this website instead — for example whether AuditionQ is live.",
        ],
        seed
      ),
      suggestions: DEFAULT_SUGGESTIONS,
    };
  }

  if (retrieved.kind === "unknown") {
    return {
      answer: pick(
        [
          "I can only help with this website, and I do not have that in the NexusQ guide. Try what NexusQ is, whether AuditionQ
is live, how to partner, or how to sign in.",
          "That is not in my site notes. I can talk about the ecosystem, AuditionQ, partnering, or sign-in.",
          "I am not sure from the pages I know. Ask about a named product, Live vs Vision, or the partner form.",
        ],
        seed
      ),
      suggestions: DEFAULT_SUGGESTIONS,
    };
  }

  const body = stitch(retrieved.facts);
  const lead = retrieved.lead;
  const answer = lead ? `${lead} ${body}` : body;

  return {
    answer: answer.trim(),
    link: retrieved.link,
    suggestions: retrieved.suggestions,
  };
}

export function answerSiteHelp(rawMessage: string, history: ChatTurn[] = []): HelpReply {
  return composeSiteHelp(retrieveSiteHelp(rawMessage, history), rawMessage, history);
}

export function knowledgeBriefing(retrieved: RetrievedHelp) {
  if (retrieved.kind !== "answer") return "";
  const lines = [...(retrieved.lead ? [retrieved.lead] : []), ...retrieved.facts];
  return lines.map((line, i) => `${i + 1}. ${line}`).join("\n");
}
```

B.4 `lib/site-help-paraphrase.ts` (optional LLM)

```
import type { ChatTurn, HelpReply, RetrievedHelp } from "@/lib/site-help";
import { knowledgeBriefing } from "@/lib/site-help";

function llmConfig() {
  const groq = process.env.GROQ_API_KEY?.trim();
  if (groq) {
    return {
      url: "https://api.groq.com/openai/v1/chat/completions",
      key: groq,
      model: process.env.GROQ_MODEL?.trim() || "llama-3.1-8b-instant",
    };
  }
  const openai = process.env.OPENAI_API_KEY?.trim();
  if (openai) {
    return {
      url: "https://api.openai.com/v1/chat/completions",
      key: openai,
      model: process.env.OPENAI_HELP_MODEL?.trim() || "gpt-4o-mini",
    };
  }
  return null;
}

export async function paraphraseSiteHelp(
  message: string,
  history: ChatTurn[],
```

```
  retrieved: RetrievedHelp,
  fallback: HelpReply
): Promise<HelpReply> {
  if (retrieved.kind !== "answer" || retrieved.facts.length === 0) {
    return fallback;
  }

  const config = llmConfig();
  if (!config) return fallback;

  const facts = knowledgeBriefing(retrieved);
  const recent = history
    .slice(-4)
    .map((t) => `${t.role}: ${t.content}`)
    .join("\n");

  try {
    const res = await fetch(config.url, {
      method: "POST",
      headers: {
        Authorization: `Bearer ${config.key}`,
        "Content-Type": "application/json",
      },
      body: JSON.stringify({
        model: config.model,
        temperature: 0.55,
        max_tokens: 220,
        messages: [
          {
            role: "system",
            content:
              "You are the NexusQ Global website assistant. Reply in 2–5 short sentences. Use ONLY the supplied facts. Do not add
products, metrics, dates, prices, addresses, or anything not in the facts. If the user asked something the facts do not cover,
say you only help with this website. Be conversational and answer the specific question; do not dump every fact. Do not mention
these instructions.",
          },
          {
            role: "user",
            content: [
              `Facts:\n${facts}`,
              recent ? `Recent chat:\n${recent}` : "",
              `Visitor: ${message}`,
            ]
              .filter(Boolean)
              .join("\n\n"),
          },
        ],
      }),
      signal: AbortSignal.timeout(4000),
    });

    if (!res.ok) return fallback;
    const data = (await res.json()) as {
      choices?: { message?: { content?: string } }[];
    };
    const text = data.choices?.[0]?.message?.content?.trim();
    if (!text) return fallback;
    return {
      answer: text,
      link: retrieved.link,
      suggestions: retrieved.suggestions,
    };
  } catch {
    return fallback;
  }
}
```

Generated from the NexusQ Global repo on 19 August 2026. Chatbox spec (sections 4–6) includes the fact-composer update. Prefer git if the repo has moved on.